

WB Pharmacy Portal — API Documentation

API Documentation

Project: WB Pharmacy Portal — Backend Services

Base URL: `http://127.0.0.1:8000/api`

Auth: All new APIs are public (`withoutMiddleware('authenticate')`) unless noted.

Content-Type: `application/json`

Table of Contents

1. Admin Details — Get by Username
 2. Generate OTP — Send by Username
 3. Generate OTP — Verify
 4. Generate OTP — Update OTP Used (Send by Contact)
 5. Evaluator — Institute Allocation Summary
-

1. Admin Details — Get by Username

Endpoint: `POST /api/admin-details/by-username`

Also accepts: `GET /api/admin-details/by-username?username=AIE&user_type_id=8`

DB Function: `public.fn_getadmindetailsbyusername(p_username, p_user_type_id)`

Auth Required: No

Request Body

Field	Type	Required	Description
<code>username</code>	string	Yes	Admin username (e.g. AIE)
<code>user_type_id</code>	integer	Yes	User type ID (e.g. 8)

Example Request

```
curl --location 'http://127.0.0.1:8000/api/admin-details/by-username' \  
--header 'Content-Type: application/json' \  
--data '{  
  "username": "AIE",
```

```
"user_type_id": 8
}]'
```

Success Response 200

```
{
  "error": false,
  "message": "Admin details fetched successfully.",
  "data": {
    "adminUserId": 1,
    "fullName": "John Doe",
    "email": "john@example.com",
    "contactNo": "9876543210",
    "userId": 8
  }
}
```

Error Responses

HTTP	Condition
422	Validation failed (missing/invalid fields)
404	Admin not found
500	DB exception

2. Generate OTP — Send by Username

Endpoint: POST /api/generate-otp/send

DB Function: public.fn_generateotp(p_username, p_usertype)

Auth Required: No

OTP Delivery Rules

user_type_id	Delivery Channel
8	Email only
9, 10, 11	SMS only
12	SMS + Email

Request Body

Field	Type	Required	Description
username	string	Yes	Admin username (e.g. AIE)
user_type_id	integer	Yes	User type ID

Example Request

```
curl --location 'http://127.0.0.1:8000/api/generate-otp/send' \
--header 'Content-Type: application/json' \
--data '{
  "username": "AIE",
  "user_type_id": 8
}'
```

Success Response 200

```
{
  "error": false,
  "message": "OTP Sent Successfully.",
  "otp_expire_time": "2026-05-14 13:02:58",
  "sent_via": {
    "sms": false,
    "email": true
  },
  "p_otp": "7777"
}
```

Note: p_otp is only returned in non-production environments for debugging. It will be null in production.

Error Responses

HTTP	Condition
422	Validation failed
400	DB function returned non-zero error code
500	DB exception

3. Generate OTP — Verify

Endpoint: POST /api/generate-otp/verify

DB Function: public.fn_getlatestotpbyusername(p_username, p_usertype)

Auth Required: No

Request Body

Field	Type	Required	Description
username	string	Yes	Admin username or contact number
user_type_id	integer	Yes	User type ID
otp	string	Yes	4-digit OTP received by user

Example Request

```
curl --location 'http://127.0.0.1:8000/api/generate-otp/verify' \
--header 'Content-Type: application/json' \
--data '{
  "username": "AIE",
  "user_type_id": 8,
  "otp": "7777"
}'
```

Success Response 200

```
{
  "error": false,
  "message": "OTP Used Successfully."
}
```

Error Responses

HTTP	Condition
422	Validation failed
404	No OTP found for this user
400	Incorrect OTP
500	DB exception

4. Generate OTP — Update OTP Used

Endpoint: POST /api/generate-otp/update-otp-used

DB Function: public.fn_updateuserotpbycontactno(p_contact_no, p_user_type, p_encotp)

Auth Required: No

Purpose: Generates OTP in PHP, stores it in DB via the function, then sends via SMS/Email based on user type.

OTP Delivery Rules

user_type_id	Delivery Channel
8	Email only
9, 10, 11	SMS only
12	SMS + Email

Request Body

Field	Type	Required	Description
contact_no	string	Yes	Phone number or email address
user_type_id	integer	Yes	User type ID

Example Requests

```
# SMS (type 9)
curl --location 'http://127.0.0.1:8000/api/generate-otp/update-otp-used' \
--header 'Content-Type: application/json' \
--data '{
  "contact_no": "7980544903",
  "user_type_id": 9
}'
```

```
# Email (type 8)
curl --location 'http://127.0.0.1:8000/api/generate-otp/update-otp-used' \
--header 'Content-Type: application/json' \
--data '{
  "contact_no": "aimed.png@gmail.com",
  "user_type_id": 8
}'
```

```
# SMS + Email (type 12)
curl --location 'http://127.0.0.1:8000/api/generate-otp/update-otp-used' \
--header 'Content-Type: application/json' \
--data '{
  "contact_no": "7980544903",
  "user_type_id": 12
}'
```

Success Response 200

```
{
  "error": false,
  "message": "OTP Sent Successfully.",
  "otp_expire_time": "2026-05-14 13:02:58",
  "sent_via": {
    "sms": true,
    "email": false
  },
  "p_otp": "1459"
}
```

Note: p_otp is only returned in non-production environments for debugging. It will be null in production.

Error Responses

HTTP	Condition
422	Validation failed
400	DB function returned non-zero error code
500	DB exception

5. Evaluator — Institute Allocation Summary

Endpoint: POST /api/evaluator/inst-allocation-summary

DB Function: public.fn_admin_getevaluatorinstallocationsummary_v1(p_admin_user_id, p_user_type_id, p_evaluator_type_id, p_exam_year, p_semester)

Auth Required: No

Request Body

Field	Type	Required	Description
admin_user_id	integer	Yes	Admin user ID
user_type_id	integer	Yes	User type ID
evaluator_type_id	integer	Yes	Evaluator type ID (e.g. 1)
exam_year	integer	Yes	Exam year (e.g. 2025)
semester	integer	Yes	Semester number (e.g. 1)

Example Request

```
curl --location 'http://127.0.0.1:8000/api/evaluator/inst-allocation-summary' \
--header 'Content-Type: application/json' \
--data '{
  "admin_user_id": 1,
  "user_type_id": 8,
  "evaluator_type_id": 1,
  "exam_year": 2025,
  "semester": 1
}'
```

Success Response 200

```
{
  "version": "1.0",
  "status": 0,
  "message": "Data fetch successfully",
  "data": {
    "internal_institutes": [
      {
        "inst_id": 1,
        "inst_code": "JCG",
        "inst_name": "JNAN CHANDRA GHOSH POLYTECHNIC",
```

```

    "pending_department": 0,
    "departments": [
      {
        "dept_id": 1,
        "dept_code": "3DAG",
        "total_subjects": 2,
        "total_pending_subjects": 1
      }
    ],
    "external_institutes": [],
    "other_institutes": []
  }
}

```

Grouping Logic

assignedEvaluatorTypeId	Response Key
1	internal_institutes
2	external_institutes
3	other_institutes

Each institute contains a `departments` array built from all rows matching that `assignedInstId`.

Error Responses

HTTP	Condition
422	Validation failed
404	No data found
500	DB exception or parse failure

Login Flow Sequence

```

Step 1 → POST /api/admin-details/by-username (fetch admin info)
Step 2 → POST /api/generate-otp/send (generate & send OTP)
Step 3 → POST /api/generate-otp/verify (verify OTP entered by user)
Step 4 → POST /api/generate-otp/update-otp-used (mark OTP as used via contact)

```

Log Files

All API calls are logged to:

services/storage/logs/laravel-YYYY-MM-DD.log

Tail live logs:

```
tail -f services/storage/logs/laravel-$(date +%Y-%m-%d).log
```